

DeployFlow: Blue-Green Deployment for ASP.NET Core on a Single VPS

For small teams running a .NET API on one server: a repeatable, health-verified release pipeline on a single Ubuntu VM — with versioned releases, controlled traffic switching through Nginx, and a one-command rollback path.

GitHub Actions

Docker

GHCR

Ubuntu / Linux

Nginx

Bash

ASP.NET Core 8

SSH / SCP

THE DELIVERY PATH

GitHub

→

GitHub Actions

→

GHCR

→

SSH

→

Ubuntu

→

Docker

→

Nginx

→

Application

A push to **main** builds and tests the application, publishes a commit-tagged image to the registry, and deploys it to the server as a candidate container. Traffic only moves once that candidate answers a health check — and the release it replaced stays on disk, ready to be restored by one command.

OBJECTIVE

Replace the common manual, SSH-driven release with an automated pipeline that verifies each version before it serves traffic and can be reversed on demand.

ENVIRONMENT

- One Ubuntu VM
- Docker, non-root containers
- Nginx reverse proxy
- GHCR image registry
- No orchestrator, no added services

BUILT

- Two-job CI/CD workflow
- Six deployment scripts
- Health-gated traffic switch
- Single-command rollback
- Automatic release retention

1 push

TRIGGERS BUILD, TEST,
PUBLISH & DEPLOY

SHA-tagged

EVERY IMAGE
TRACEABLE TO A
COMMIT

3

RELEASES RETAINED
ON DISK FOR
ROLLBACK

1 command

REVERT TO THE
PREVIOUS RELEASE

Many small teams run a containerised .NET API on a single VPS and release it by hand over SSH: pull the code, rebuild the image, stop the running container, start the new one, then hope it comes up. That process is fast to start with and expensive to live with. DeployFlow is a personal reference implementation built to show how to replace it.

Manual deployment is the only path to production

Each release depends on a person typing the right sequence of Docker commands in the right order on the right server. Steps are remembered rather than recorded, so the deployment procedure lives in one engineer's head — and cannot be handed over, audited, or run twice identically.

Releases are not repeatable or traceable

Images are built on the server from whatever the working tree happens to contain. Nothing ties a running container back to a specific commit, so the question “*which code is live right now?*” has no reliable answer.

No verification before traffic is switched

The old container is stopped before the new one has proven it can serve requests. A bad build, a missing environment variable, or a start-up exception becomes a live outage, discovered by a user or by someone checking the logs.

No way back

Once the previous container is removed, recovery means rebuilding the old code from scratch under pressure. There is no known-good artefact standing by, and no defined rollback procedure.

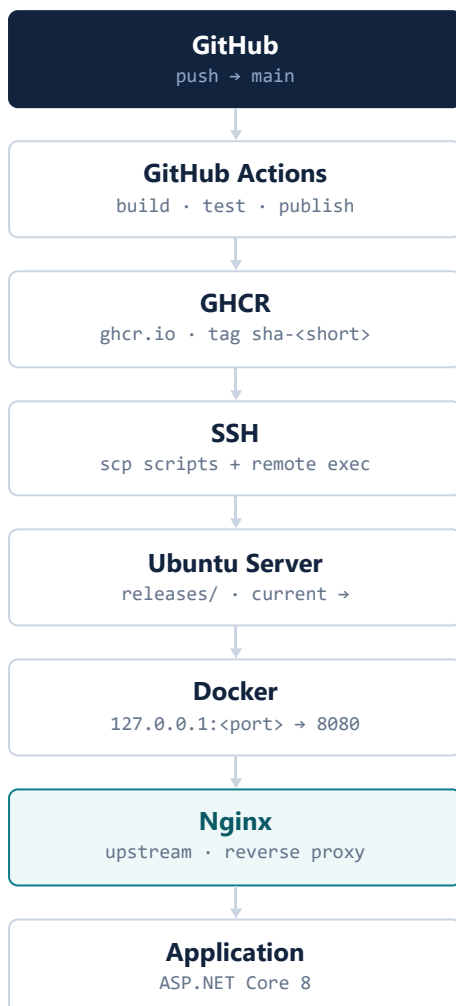
Kubernetes is the wrong size for the problem

A managed orchestrator solves this, but for one API on one VM it adds a control plane, a cost line, and an operational skill set out of proportion to the workload.

What the solution had to deliver

- Deployment triggered by `git push`, with no manual server steps
- Every artefact tagged with its Git commit SHA
- The new version proven healthy *before* it receives traffic
- A failed release must leave the running version untouched
- Previous releases kept on disk, ready to be restored
- Rollback as a single, scripted command
- No orchestrator, no extra managed infrastructure
- Registry credentials never persisted on the server

One path from commit to served request. CI builds and publishes the artefact; the server pulls it and decides whether it earns traffic.



GitHub	Single trigger: a push to <code>main</code> . The repository is the only place a release starts, so the pipeline is the audit trail.
GitHub Actions	Restores, builds and runs tests on .NET 8, then builds the image from a multi-stage Dockerfile. A failing test stops the run before anything is published.
GHCR	The image is pushed to GitHub Container Registry tagged <code>sha- <short-sha></code> and <code>latest</code> . The SHA tag is what gets deployed — so the running container maps back to an exact commit.
SSH	The deploy job copies the <code>scripts/</code> directory to the server via SCP, then executes the orchestrator over SSH using key-based auth. Registry login uses a token passed in at run time and is wiped with <code>docker logout</code> when the run ends.
Ubuntu	A Capistrano-style layout: each deploy gets its own timestamped directory under <code>releases/</code> , and a <code>current</code> symlink names the active one. Release metadata (port, container, image) is written to disk so later scripts are stateless.
Docker	The new version starts as a <i>candidate</i> container on a random high port bound to <code>127.0.0.1</code> only — reachable for health checks, never from the internet. Runs as a non-root user with <code>--restart unless-stopped</code> .
Nginx	The single public entry point, serving HTTPS with a Let's Encrypt certificate and redirecting HTTP to it. Its <code>upstream</code> block lives in one generated include file; rewriting that file and reloading Nginx is the only action that moves traffic between versions.
Application	Exposes <code>/health</code> as the readiness gate, plus <code>/version</code> , <code>/environment</code> and <code>/uptime</code> so the live deployment can be verified from outside.

DESIGN DECISION

Nothing is built on the production server and no source is copied there — only signed-in image pulls and small Bash scripts. The server's job is to run a verified artefact and route traffic to it, which keeps its state minimal and its failure modes few.

1

Build `GitHub Actions runner`

Checkout, set up .NET 8, `dotnet restore`, `dotnet build -c Release`, then `dotnet test`. Tests run against the release build, so the same configuration that ships is the one that was verified. The image is then built from a multi-stage Dockerfile: SDK image compiles and publishes, and only the published output is copied into a slim `aspnet:8.0-alpine` runtime layer that runs as a non-root user.

2

Publish `GHCR`

Login uses the workflow's scoped `GITHUB_TOKEN` with `packages: write`. Tags are generated from the commit (`type=sha`) alongside `latest`, and the image is pushed to `ghcr.io/<owner>/<repo>/sample-api`. From here on, the release is an immutable artefact identified by its commit — not a build recipe.

3

Deploy `SSH → Ubuntu`

A second job (gated on a successful build) sparse-checks-out only `scripts/`, SCPs it to the app directory, and runs `deploy.sh` over SSH with the fully qualified image reference. On the server the script creates a timestamped release directory, pulls the image, and starts a candidate container on a random loopback-bound port — recording its port, container name and image tag inside the release directory. The currently live container keeps serving throughout.

4

Verify `health-check.sh`

`health-check.sh` polls `http://127.0.0.1:<port>/health` with `curl -f`, up to 10 attempts at 3-second intervals, giving a start-up window of roughly 30 seconds. Only a `200 OK` allows the release to continue. If the window expires, the candidate container is stopped and removed, its release directory is deleted, and the job exits non-zero — a red build instead of a broken site.

5

Release management `switch-release.sh · cleanup.sh`

Once healthy, the Nginx upstream include is rewritten to the candidate's port, `nginx -t` validates the configuration, and `nginx -s reload` applies it. The `current` symlink is repointed atomically with `ln -sfn`. After a 10-second drain for in-flight requests, the previous container is stopped but kept for rollback, and `cleanup.sh` trims the release history to the three most recent, removing their containers and pruning dangling images to reclaim disk.

```
# .github/workflows/deploy.yml – the deploy job, in essence
scp  scripts/          → ~/apps/sample-api/scripts/
ssh  docker login ghcr.io -u $ACTOR --password-stdin
      IMAGE=ghcr.io/$OWNER/$REPO/sample-api:sha-${SHA:0:7}
      bash ~/apps/sample-api/scripts/deploy.sh "$IMAGE"
      docker logout ghcr.io          # credentials never left behind
```

Four mechanisms carry the reliability story: a health gate before traffic moves, immutable release directories, a symlink that names the active release, and a retained previous release to fall back to.

Health verification is the gate

The candidate is only reachable on loopback until it passes. The switch step is never reached on a failed check — the deploy aborts with the live version still serving. Failure surfaces as a failed pipeline run, not as user-visible errors.

Release directories are immutable

Each deploy writes a fresh `releases/<timestamp>/` holding the port, container name and image reference. Nothing is edited in place, so a release is a self-describing record that later scripts can act on without shared state.

The `current` symlink is the source of truth

In `-sfn` replaces the symlink in a single filesystem operation — there is no moment where `current` points at nothing. Any script, or any operator, can read it to learn which release is live.

Previous releases are retained

`cleanup.sh` keeps the three most recent releases. The prior release's directory and metadata survive the switch, so a known-good version is always documented and ready to be brought back without a rebuild.

On-disk layout

```
~/apps/sample-api/
├ releases/
│   ├── 20260731-101204/ retained
│   ├── 20260803-084417/ rollback target
│   └ 20260811-131955/ live
│       ├── .port 48211
│       ├── .container sample-api_20260811-131955
│       └ .image ghcr.io/...:sha-f06d7cc
├ current → releases/20260811-131955
└ scripts/
```

Rollback approach

- Run `rollback.sh` on the server — one command, no rebuild, no pipeline run
- It reads `current` to find the live release, then takes the release before it
- That release's stopped container is started again; if it is gone or its port is taken, it is recreated from the release's recorded image on a free port
- The *same* health check must pass on the previous release before traffic moves back
- The Nginx upstream is rewritten, validated with `nginx -t` and reloaded
- `current` is repointed to the previous release
- After a drain period, the faulty container and its release directory are removed, so the bad version cannot be restarted by accident

STATED HONESTLY

This is not a claim of guaranteed zero downtime. The design goal is **repeatable releases with controlled traffic switching and rollback support**: no version receives traffic until it answers a health check, the switch is a validated Nginx reload rather than a stop-then-start, the old container gets a drain period before it stops, and a verified previous release stays on disk. Actual downtime depends on the application's own shutdown behaviour; `measure-downtime.sh` measures it from the client side during a real switch.

Push → live

Fully automated: build, test, publish, deploy, verify — no manual server steps

Commit-traceable

Every running container maps to an exact SHA, checkable via `/version`

Gated

Traffic switches only after a `200 OK` health check on the new version

Reversible

One command restores the previous release, health-checked before the switch

Self-limiting

Release history trimmed to 3 and dangling images pruned each deploy

No orchestrator

One Ubuntu VM, Docker and Nginx — no control plane, no added cost

TYPICAL MANUAL SETUP

- Hand-typed SSH commands, remembered not recorded
- Images built on the production server
- No answer to “which commit is live?”
- Old container stopped before the new one is proven
- A bad build becomes a live outage
- Recovery means rebuilding under pressure

WITH DEPLOYFLOW

- A push to `main` is the whole procedure
- Immutable, SHA-tagged images pulled from GHCR
- `/version` and `current` both answer it
- Candidate proves itself before traffic moves
- A bad build fails the pipeline, not production
- Previous release stays on disk, ready to restore

Transferable to any containerised service

COMPONENT	WHAT IT IS WORTH ON THE NEXT PROJECT
CI pipeline	Same shape for any language — only the build and test steps change.
Deploy orchestrator	Release-directory + candidate-container pattern needs only a readiness endpoint and a port.
Traffic switching	A generated, validated Nginx upstream include — portable to any service behind a proxy.
Rollback & retention	Retained releases with recorded metadata give a scripted way back, with no rebuild.
Secrets handling	Key-based SSH and run-time registry tokens — no long-lived credentials on the host.

Where this goes next

- Automatic rollback on post-switch error-rate checks
- Switch downtime measured under sustained load and published

ABOUT THIS CASE STUDY

A personal project, built solo as a working reference implementation, not a diagram — every mechanism described here exists as committed workflow YAML and Bash on the server.

Live: <https://deployflow.inventstarts.com/version> · Source: github.com/Asharkhalid/deployflow